

**CSE/ISyE 6740**  
**Computational Data Analysis**

# **Gradient Descent**

10/07/2026

Kai Wang, Assistant Professor in Computational Science and Engineering

[kwang692@gatech.edu](mailto:kwang692@gatech.edu)

# Outline

- **Gradient Decent**
  - First-order gradient descent
  - Second-order gradient descent (Newton's method)
  - Momentum methods
  - L1 and L2 regularization

# Gradient Descent

# Why Should We Care About Gradient Descent?

- Most of the optimization problems in machine learning are non-convex.
  - Maximum likelihood estimation
  - Neural networks
- **Gradient descent** is a powerful iterative method to solve **convex/non-convex optimization** problems **computationally efficiently**.
- All neural network training methods are based on gradient descent.

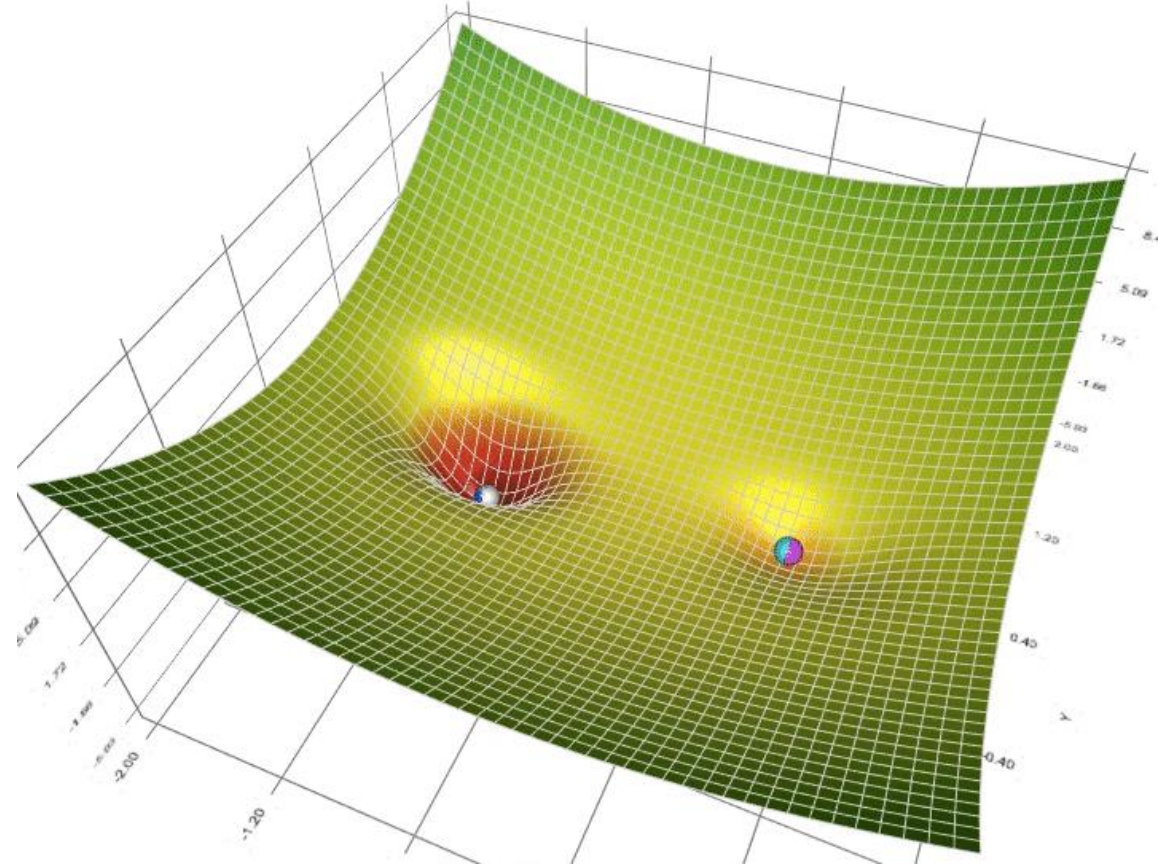


Photo credit: [Lili Jiang on Medium](#)

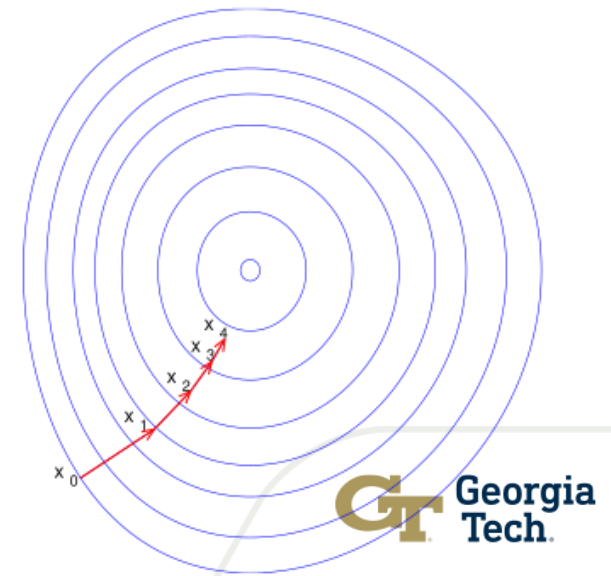
# Gradient Descent

- Gradient descent is an important iterative method to solve minimization problem.

$$\min_{x \in X} f(x)$$

- Gradient descent runs as the following update with learning rate  $\gamma > 0$

$$x^{t+1} = x^t - \gamma \nabla_x f(x^t)$$



# Why is the Direction $-\nabla_x f(x)$ ?

- At the point  $x$  with function value  $f(x)$ , how do we know which direction  $\delta$  to move to reduce the function value the most?
- **Taylor expansion:**  $x, y \in \mathbb{R}$  and  $\delta = y - x$

$$f(y) = f(x) + \frac{1}{1!} f'(x)(y - x) + \frac{1}{2!} f''(x)(y - x)^2 + \frac{1}{3!} f'''(x)(y - x)^3 + \dots = \sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} (y - x)^k$$

$$f(x + \delta) = f(x) + \frac{1}{1!} f'(x)\delta + \frac{1}{2!} f''(x)\delta^2 + \frac{1}{3!} f'''(x)\delta^3 + \dots = \sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} \delta^k$$

# Why is the Direction $-\nabla_x f(x)$ ?

- **Taylor expansion (multi-variable function):** for  $x, \delta \in \mathbb{R}^d$

$$f(x + \delta) = f(x) + \frac{1}{1!} \nabla_x f(x)^\top \delta + \frac{1}{2!} \delta^\top \nabla_x^2 f(x) \delta + \dots = \sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} [\delta, \delta, \dots, \delta]$$

- We will only use the 1<sup>st</sup> and 2<sup>nd</sup> order Taylor expansion

$$f(x + \delta) = f(x) + \nabla_x f(x)^\top \delta + o(\|\delta\|)$$

# Why is the Direction $-\nabla_x f(x)$ ?

- Given the 1<sup>st</sup> order Taylor expansion:

$$f(x + \delta) = f(x) + \nabla_x f(x)^\top \delta + o(\|\delta\|) \approx f(x) + \nabla_x f(x)^\top \delta$$

- Now if we want to find a direction  $\delta$  to move from  $x$  to  $x + \delta$ , which direction gives us the largest descent?

$$\min_{\delta: \|\delta\| < r} f(x + \delta) \quad \Leftrightarrow \quad \min_{\delta: \|\delta\| < r} f(x) + \nabla_x f(x)^\top \delta$$

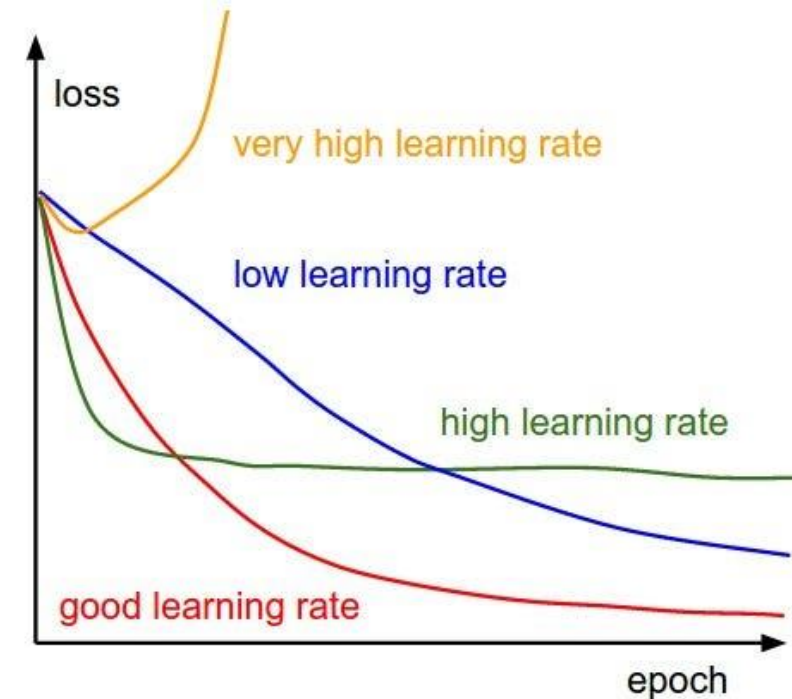
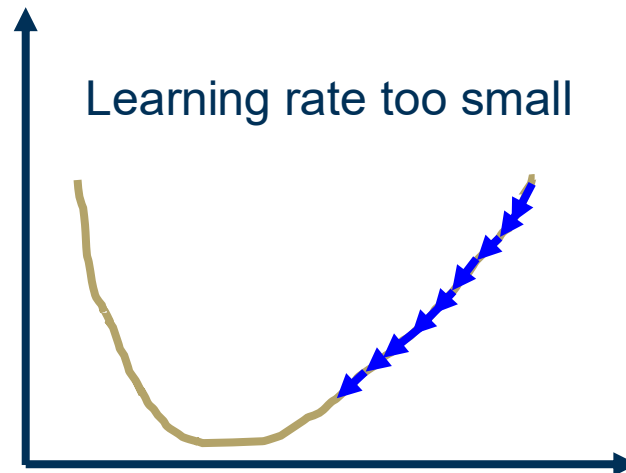
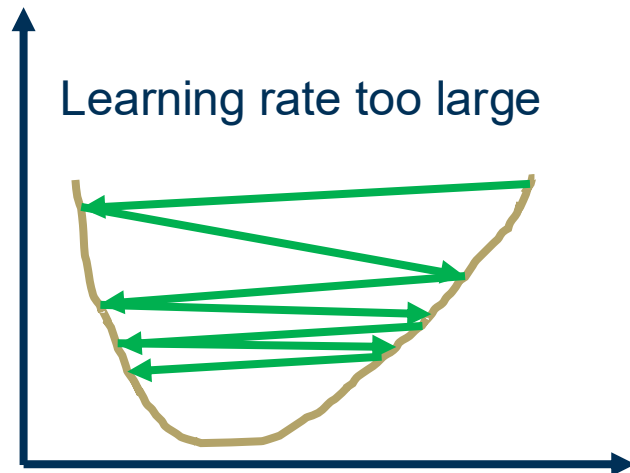
- Therefore, we should pick  $\delta = -\nabla_x f(x)$ 
  - When we want to maximize, we should pick  $\delta = \nabla_x f(x)$



# Gradient Descent

- Gradient descent runs as the following update with **learning rate  $\gamma > 0$**

1. Compute gradient  $g^t = \nabla_x f(x^t)$
2. Update  $x^{t+1} = x^t - \gamma \nabla_x f(x^t)$



# Stochastic/Mini-batch Gradient Descent

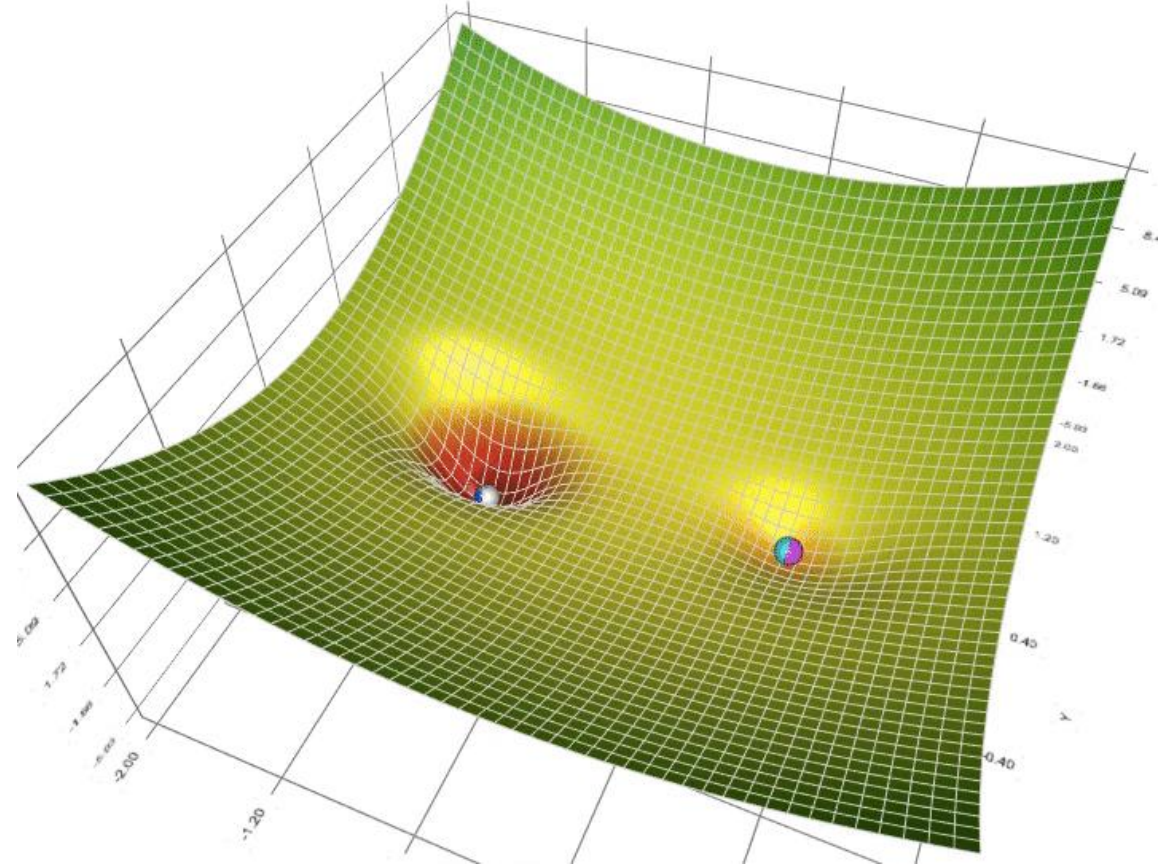
- Why stochastic gradient descent?
- What if your objective function is defined in expectation?

$$\min_{\theta} f(\theta) := \mathbb{E}_{(x,y) \sim D_{train}} [\ell(m_{\theta}(x), y)]$$

- **Full-batch gradient descent:**  $\nabla_{\theta} f(\theta) = \mathbb{E}_{(x,y) \sim D_{train}} [\nabla_{\theta} \ell(m_{\theta}(x), y)]$ 
  - Computation cost:  $O(|D_{train}| \dim \theta)$
  - Too inefficient
- **Stochastic/mini-batch gradient descent:**  $\nabla_{\theta} f(\theta) \approx \mathbb{E}_{(x,y) \sim B} [\nabla_{\theta} \ell(m_{\theta}(x), y)]$ 
  - Computation cost:  $O(|B| \dim \theta)$
  - More efficient!! Parallelization to reduce cost to  $O(\dim \theta)$ .

# Escape Local Minimum

- Stochastic gradient descents and momentum can help you escape local minimums.
- Some randomness is still needed in gradient descent algorithms.



# Second-Order Method

# Newton's Method (Second-Order Method)

- **Taylor expansion (multi-variable function):** for  $x, \delta \in \mathbb{R}^d$

$$f(x + \delta) = f(x) + \frac{1}{1!} \nabla_x f(x)^\top \delta + \frac{1}{2!} \delta^\top \nabla_x^2 f(x) \delta + \dots = \sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} [\delta, \delta, \dots, \delta]$$

- Can we extend from the 1<sup>st</sup> order to the 2<sup>nd</sup> order Taylor expansion?

$$f(x + \delta) = f(x) + \nabla_x f(x)^\top \delta + o(\|\delta\|)$$

$$f(x + \delta) = f(x) + \nabla_x f(x)^\top \delta + \frac{1}{2} \delta^\top \nabla_x^2 f(x) \delta + o(\|\delta\|^2)$$

# Newton's Method (Second-Order Method)

- Given the 2<sup>nd</sup> order Taylor expansion:

$$f(x + \delta) \approx f(x) + \nabla_x f(x)^\top \delta + \frac{1}{2} \delta^\top \nabla_x^2 f(x) \delta$$

- Now if we want to find a direction  $\delta$  to move from  $x$  to  $x + \delta$ , which direction gives us the largest descent?

$$\min_{\delta} f(x + \delta) \quad \Leftrightarrow \quad \min_{\delta} f(x) + \nabla_x f(x)^\top \delta + \frac{1}{2} \delta^\top \nabla_x^2 f(x) \delta$$

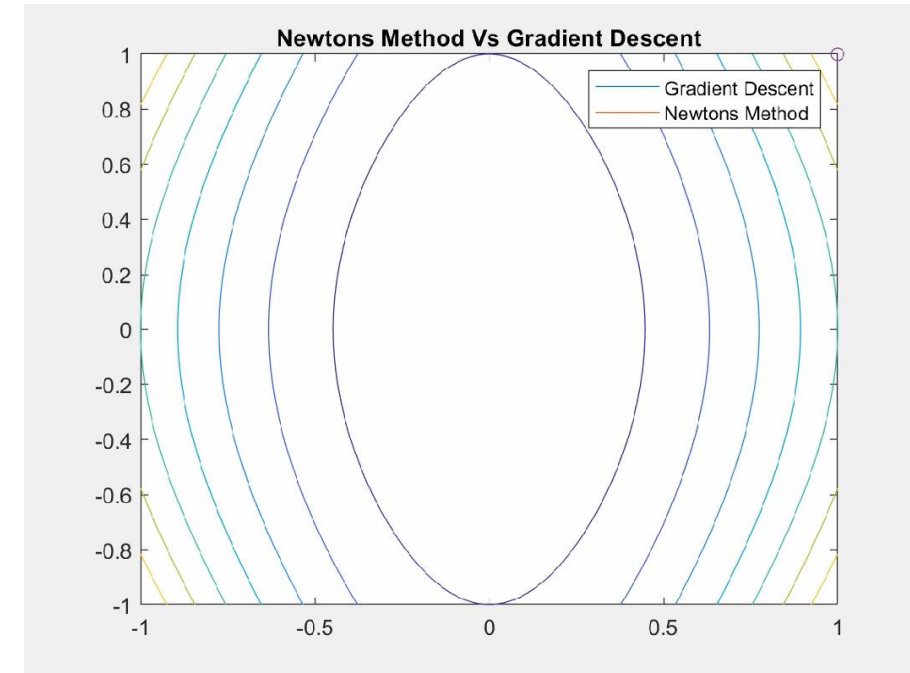
- We can minimize this quadratic problem by setting the gradient (w.r.t.  $\delta$ ) to 0
  - $\nabla_x f(x) + \nabla_x^2 f(x) \delta = 0 \quad \Rightarrow \quad \delta = -(\nabla_x^2 f(x))^{-1} \nabla_x f(x)$

# Newton's Method (Second-Order Method)

- Newton method runs as the following update:

1. Compute gradient  $g^t = \nabla_x f(x^t)$
2. Compute **the Hessian**  $\nabla_x^2 f(x^t)$
3. Update  $x^{t+1} = x^t - (\nabla_x^2 f(x^t))^{-1} \nabla_x f(x^t)$

- $x^{t+1} = x^t - (\nabla_x^2 f(x))^{-1} \nabla_x f(x)$ 
  - Computation cost:  $O(d^3)$ , or  $O(d^\omega)$  if you a fancier algorithm where  $\omega \approx 2.376$  is the best-known matrix inversion complexity
  - Memory cost:  $O(d^2)$



# Momentum Methods



# Other Gradient Descent Methods

- Gradient descent:  $x^{t+1} = x^t - \gamma \nabla_x f(x^t)$
- Newton's method:  $x^{t+1} = x^t - (\nabla_x^2 f(x))^{-1} \nabla_x f(x)$
- **AdaGrad family**
- **Gradient descent with momentum**

## Algorithms

Adadelta

Implements Adadelta algorithm.

Adafactor

Implements Adafactor algorithm.

Adagrad

Implements Adagrad algorithm.

Adam

Implements Adam algorithm.

AdamW

Implements AdamW algorithm.

SparseAdam

SparseAdam implements a masked version of the Adam algorithm suitable for sparse gradients.

Adamax

Implements Adamax algorithm (a variant of Adam based on infinity norm).

# AdaGrad (Adaptive Gradient Descent)

- AdaGrad runs as the following update with learning rate  $\gamma > 0$

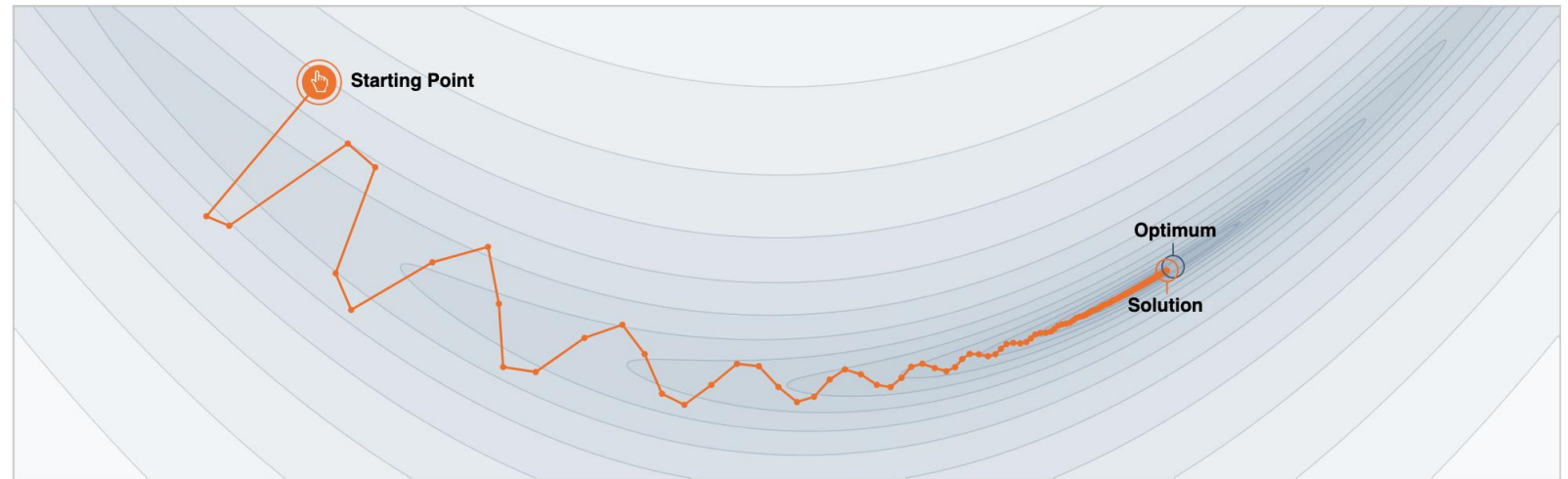
1. Compute gradient  $g^t = \nabla_x f(x^t)$
2. Compute **the adaptation term**  $G^t = \sum_{\tau=1}^t g^\tau (g^\tau)^\top$   
(think of this as the normalization term in Newton method)
3. Update  $x^{t+1} = x^t - \gamma (G^t)^{-\frac{1}{2}} \nabla_x f(x^t)$

- In practice, we often take  $g^t (g^t)^\top = \begin{bmatrix} g_1^t \\ g_2^t \\ \vdots \\ g_d^t \end{bmatrix} [g_1^t, g_2^t, \dots, g_d^t] \approx \begin{bmatrix} (g_1^t)^2 & 0 & \dots & 0 \\ 0 & (g_2^t)^2 & & 0 \\ \dots & & \dots & \\ 0 & 0 & & (g_d^t)^2 \end{bmatrix}$

- Other variants: AdaDelta, RMSprop

# Momentum Methods

- Momentum method runs as the following update with learning rate  $\gamma > 0$ 
  1. Compute gradient  $g^t = \nabla_x f(x^t)$
  2. Compute **the momentum**  $z^t = \beta z^{t-1} + \nabla_x f(x^t)$ .  
(think of this as the "momentum" of the movement, discounted by  $\beta$ , like friction)
  3. Update  $x^{t+1} = x^t - \gamma z^t$



<https://distill.pub/2017/momentum/>

# Adam (Adaptive Moment Estimate)

- Why not both?
- Adam combines AdaGrad and momentum method!

**Algorithm 1:** *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation.  $g_t^2$  indicates the elementwise square  $g_t \odot g_t$ . Good default settings for the tested machine learning problems are  $\alpha = 0.001$ ,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$  and  $\epsilon = 10^{-8}$ . All operations on vectors are element-wise. With  $\beta_1^t$  and  $\beta_2^t$  we denote  $\beta_1$  and  $\beta_2$  to the power  $t$ .

**Require:**  $\alpha$ : Stepsize

**Require:**  $\beta_1, \beta_2 \in [0, 1)$ : Exponential decay rates for the moment estimates

**Require:**  $f(\theta)$ : Stochastic objective function with parameters  $\theta$

**Require:**  $\theta_0$ : Initial parameter vector

$m_0 \leftarrow 0$  (Initialize 1<sup>st</sup> moment vector)

$v_0 \leftarrow 0$  (Initialize 2<sup>nd</sup> moment vector)

$t \leftarrow 0$  (Initialize timestep)

**while**  $\theta_t$  not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$  (Get gradients w.r.t. stochastic objective at timestep  $t$ ) **Momentum**

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$  (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$  (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  (Compute bias-corrected second raw moment estimate) **Adaptation**

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  (Update parameters)

**end while**

**return**  $\theta_t$  (Resulting parameters)

**Normalized the momentum by adaptation term**





# Comparison

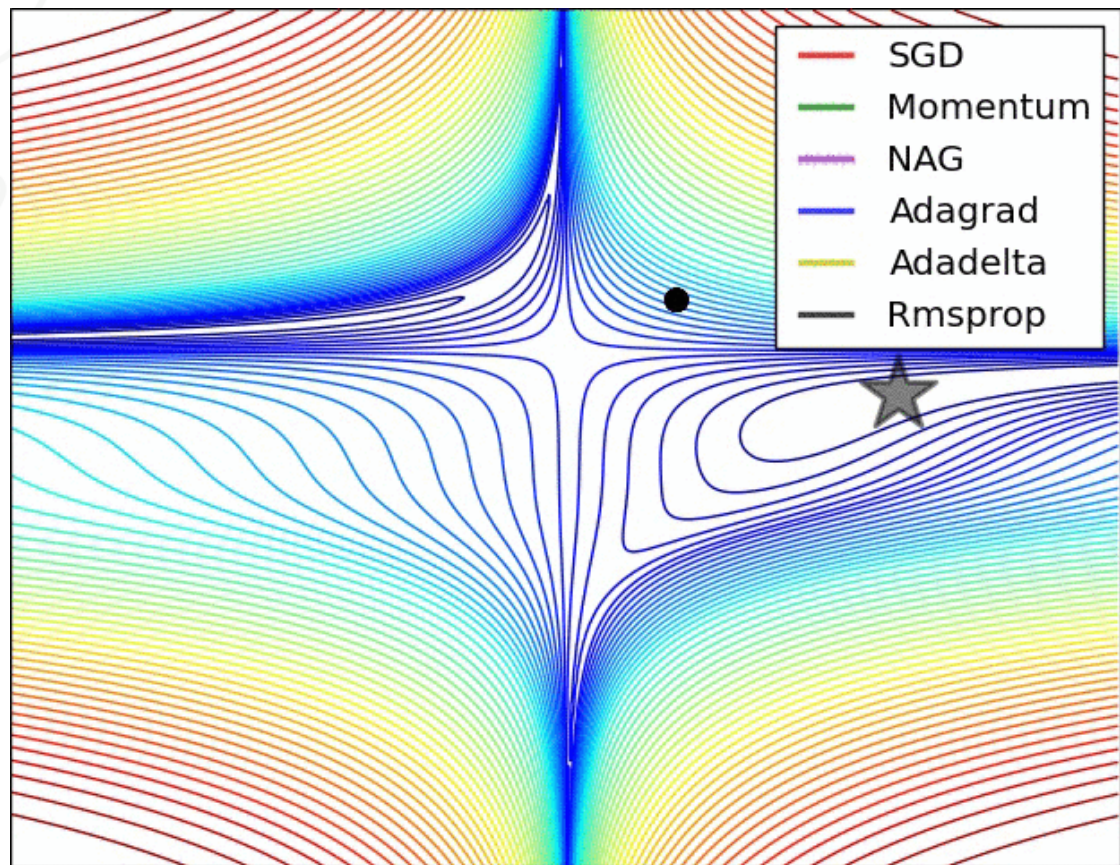
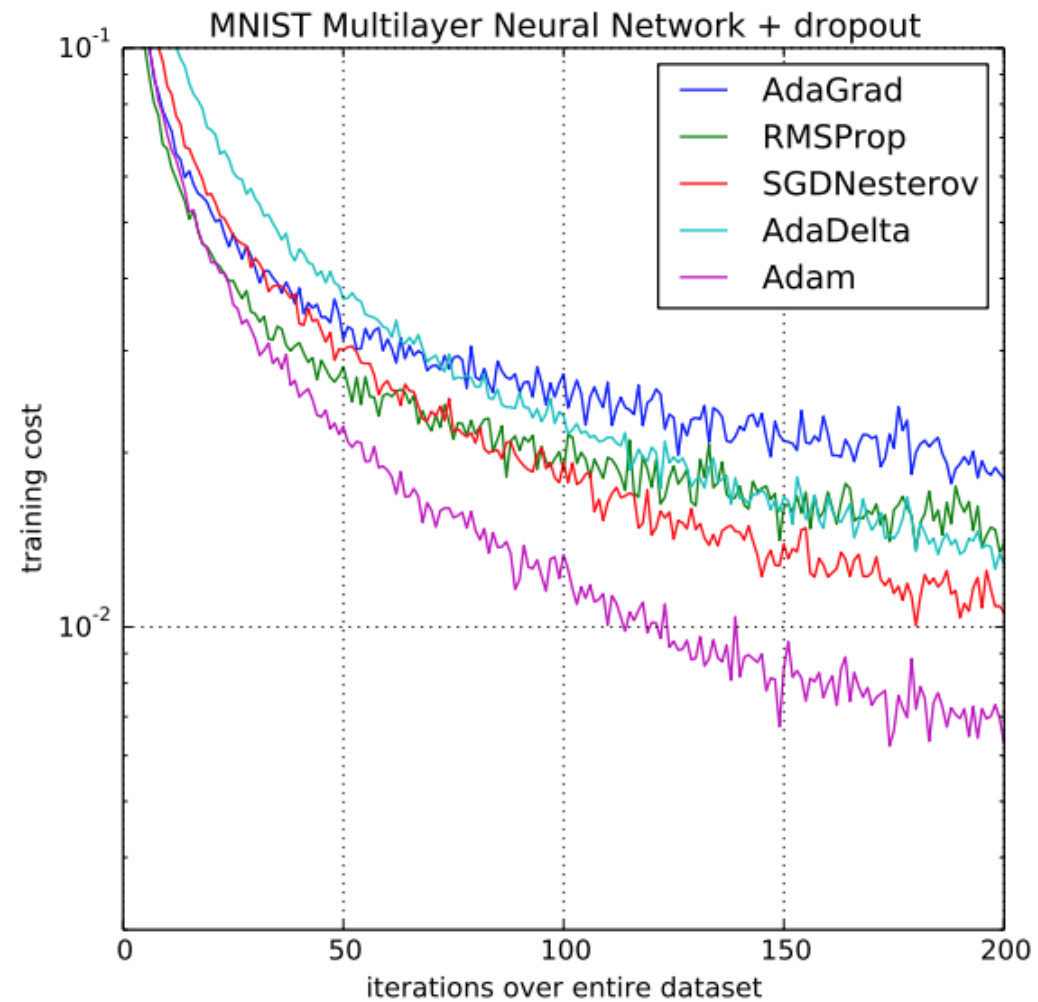


Figure credit: Oren Milman



<https://optimization.cbe.cornell.edu/index.php?title=Adam>

# PyTorch Gradient Descent

- PyTorch has implemented all these gradient descent methods for you.
- Pick one `torch.optim.Optimizer` to determine how you want to run gradient descent.
- `Optimizer.step()` will handle gradient descent for you.
- Remember to run `optimizer.zero_grad()` to reset the gradient

```
CLASS torch.optim.Optimizer(params, defaults) \[SOURCE\]
```

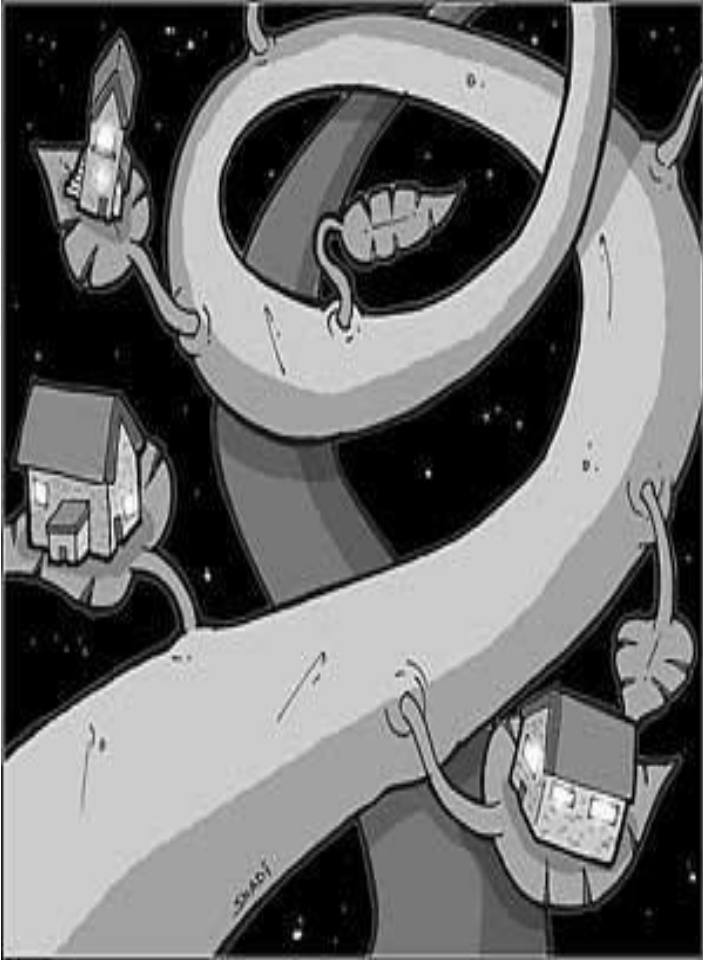
```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

```
# Set the model to training mode - important for batch normalization and dropout layers
# Unnecessary in this situation but added for best practices
model.train()
for batch, (X, y) in enumerate(dataloader):
    # Compute prediction and loss
    pred = model(X)
    loss = loss_fn(pred, y)

    # Backpropagation
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

# L1 and L2 Regularization

# Linear Regression



- Suppose you want to move to Atlanta, and you want to find the most **reasonably priced** apartment satisfying your **needs** (I know it is hard and probably too late):

| Living area (ft <sup>2</sup> ) | # bedroom | Monthly rent (\$) |
|--------------------------------|-----------|-------------------|
| 230                            | 1         | 900               |
| 506                            | 2         | 1800              |
| 433                            | 2         | 1500              |
| 190                            | 1         | 800               |
| ...                            |           |                   |
| 150                            | 1         | ?                 |
| 270                            | 1.5       | ?                 |



# Matrix Version of the Gradient

- Given feature  $X = (x^1, x^2, \dots, x^n)^\top \in \mathbb{R}^{n \times d}$  and labels  $y = (y^1, y^2, \dots, y^n)^\top \in \mathbb{R}^n$ , find  $\theta$  that minimizes the mean square error

$$\hat{\theta} = \operatorname{argmin}_{\theta} L(\theta) := \frac{1}{n} \sum_{i=1}^n (y^i - \theta^\top x^i)^2$$

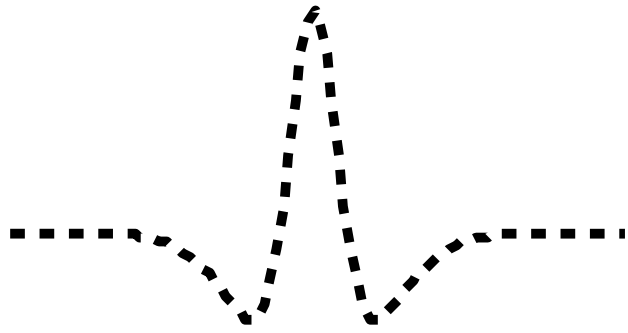
- We can write this in matrix form and find  $\hat{\theta}$  by setting the gradient to be zero:

$$\begin{aligned} \frac{\partial L(\theta)}{\partial \theta} &= -\frac{2}{n} X^\top y + \frac{2}{n} X^\top X \theta = 0 \\ \Rightarrow \hat{\theta} &= (X^\top X)^{-1} X^\top y \end{aligned}$$

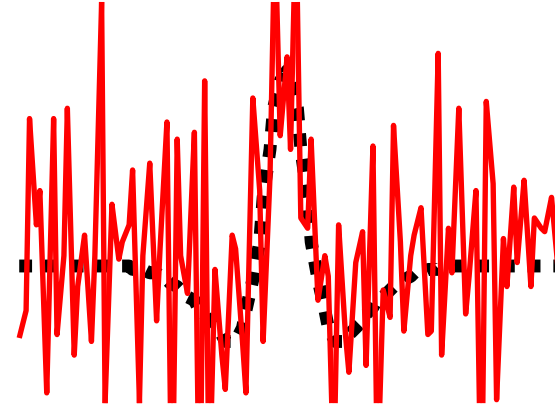
- What if  $X^\top X$  is nearly singular?

# Linear Regression

True curve



$$\hat{\theta} = (X^T X)^{-1} X^T y$$



# Ridge Regression

- Given  $n$  data points, find  $\theta$  that minimizes the regularized mean square error

$$\theta^{rr} = \operatorname{argmin}_{\theta} L(\theta) := \frac{1}{n} \left( \sum_{i=1}^n (y^i - \theta^{\top} x^i)^2 + \lambda \|\theta\|^2 \right)$$

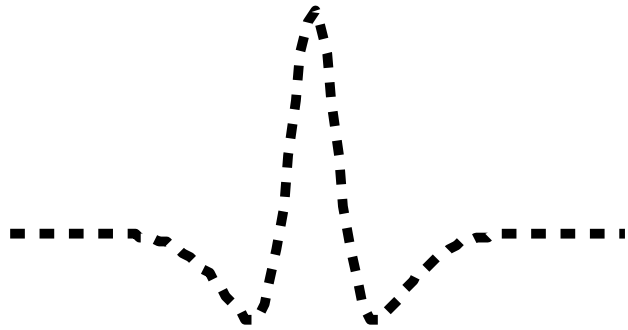
- Gradient becomes

$$\begin{aligned} \frac{\partial L(\theta)}{\partial \theta} &= -\frac{2}{n} X^{\top} y + \frac{2}{n} X^{\top} X \theta + \frac{2\lambda}{n} \theta = 0 \\ \Rightarrow \theta^{rr} &= (X^{\top} X + \lambda I)^{-1} X^{\top} y \end{aligned}$$

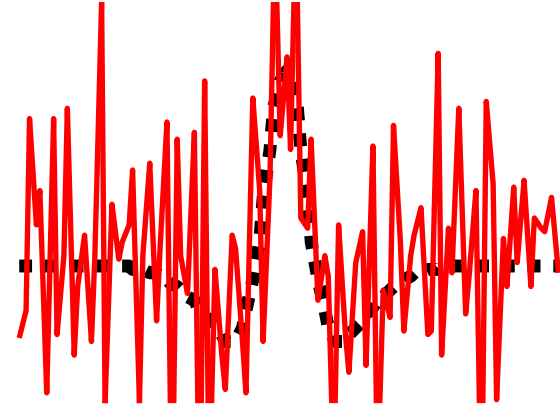
If we choose a different  $\lambda$ , the solution will be different.

# Ridge Regression

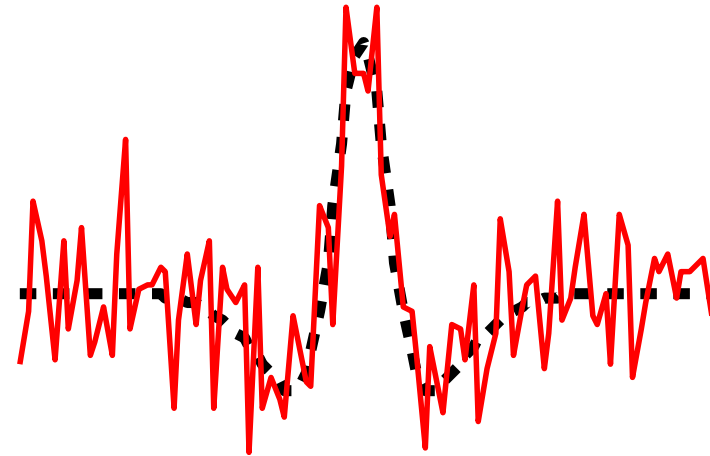
True curve



$$\hat{\theta} = (X^T X)^{-1} X^T y$$



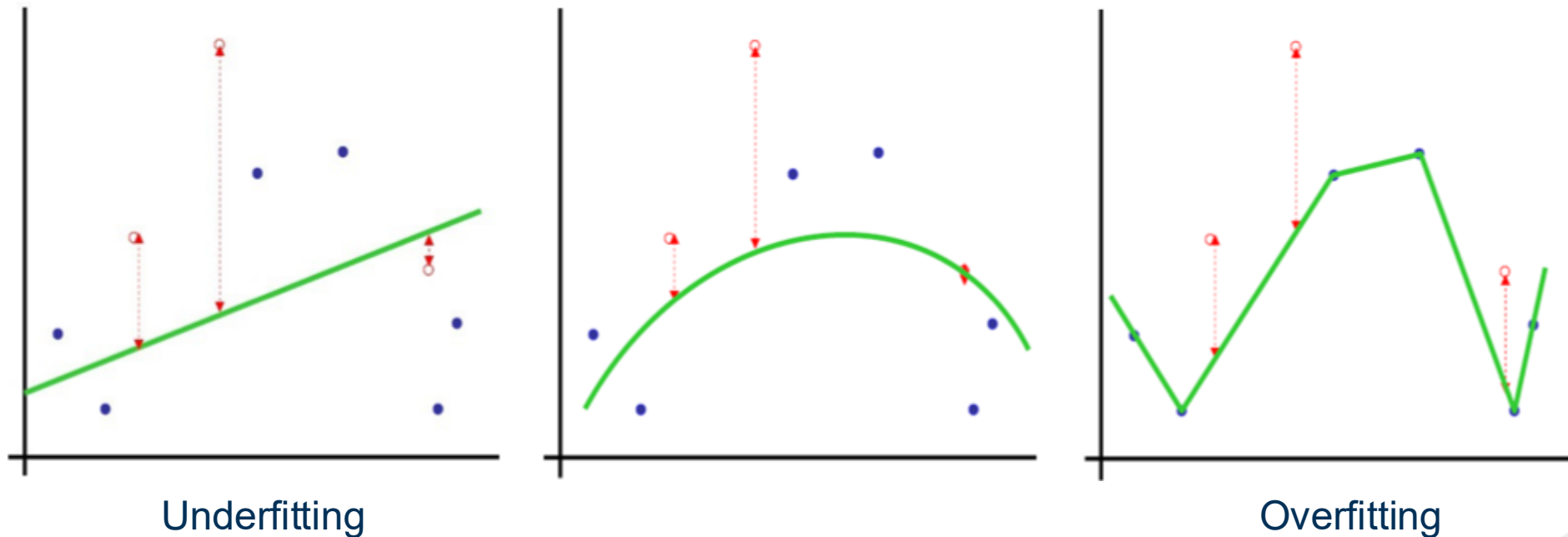
$$\hat{\theta} = (X^T X + \lambda I)^{-1} X^T y$$



- **L2 (a.k.a. “ridge”)** regularization:
  - Penalize squared parameters
  - **Suppress all features (larger weights)**

# Overfitting/Underfitting

- **Blue points:** training data points
- **Red points:** test data points
- The fit in the middle panel achieves a balance of small error in both training and test points.

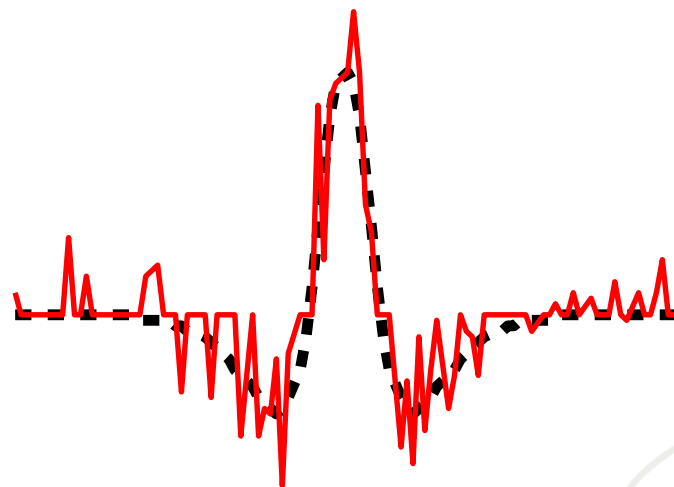


# Lasso Regression

- Given  $n$  data points, find  $\theta$  that minimizes the regularized mean square error.

$$\theta^{lasso} = \operatorname{argmin}_{\theta} L(\theta) := \frac{1}{n} \left( \sum_{i=1}^n (y^i - \theta^{\top} x^i)^2 + \lambda \|\theta\|_1 \right)$$

- L1** (a.k.a., “**LASSO**”, Least Absolute Shrinkage and Selection Operator) regularization:
  - Penalize the absolute value of parameters
  - Encourage sparse parameters**
  - Select informative features
- No closed-form solution  $\Rightarrow$  use gradient descent



# Comparison Between L1 and L2 Regularization

| Feature                  | L1 Regularization (Lasso)                                          | L2 Regularization (Ridge)                                    |
|--------------------------|--------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Penalty term</b>      | Sum of absolute values $\ \theta\ _1$                              | Sum of squares $\ \theta\ ^2$                                |
| <b>Effect on weights</b> | <b>Sparse</b> weights                                              | <b>Smaller</b> weights                                       |
| <b>Feature selection</b> | <b>Yes</b> (pick the ones with large weights)                      | <b>No</b> (all of the weights are usually small)             |
| <b>When to use</b>       | Useful when you expect sparse features or want feature selection   | Useful when you expect many relevant or correlated features  |
| <b>Stability</b>         | Can be <b>unstable</b> with highly correlated features             | <b>More stable</b> , especially with noisy or small datasets |
| <b>Interpretability</b>  | Can lead to simpler, more interpretable models with fewer features | Less interpretable due to many non-zero features             |



# Questions

- How to apply gradient descent to  $\min_{x \in X} f(x)$  with L2 regularization?
- How to apply gradient descent to  $\min_{x \in X} f(x)$  with L1 regularization?
- How to implement gradient descent with L1 or L2 regularization on neural networks?